ARES CTI Dashboard

Cyber Threat Intelligence Aggregation Platform

Technical Documentation & Architecture Reference

Author	Rafael Rodriguez
Stack	Python / Flask React / TypeScript / Vite
Date	April 2026
Version	1.0.0

Aggregates threat intelligence from **AbuseIPDB**, **AlienVault OTX**, and **VirusTotal**
into a unified analyst-facing dashboard with IOC lookup, scoring, and live feeds.

Contents

1 Executive Summary **3**

2 What the App Does **4**
2.1 Core Features . . . 4
2.1.1 IOC Lookup . . . 4
2.1.2 Threat Feed . . . 4
2.1.3 Dashboard Overview . . . 4
2.1.4 Bulk Lookup (API-first) . . . 4

3 System Architecture **5**
3.1 High-Level Architecture . . . 5
3.2 Backend Architecture – Flask Blueprints . . . 5
3.3 Frontend Architecture – React Component Tree . . . 6
3.4 IOC Lookup Data Flow . . . 7
3.5 Caching Strategy . . . 8

4 Design Choices & Rationale **9**
4.1 Backend: Why Flask? . . . 9
4.2 Parallelisation: ThreadPoolExecutor . . . 9
4.3 Score Normalisation . . . 9
4.4 Frontend: Why Vite + React? . . . 9
4.5 The Proxy Pattern . . . 9
4.6 App Factory Pattern (Flask) . . . 10

5 API Reference **11**
5.1 Endpoint Summary . . . 11
5.2 Indicator Response Schema . . . 11

6 Security Review **12**
6.1 Current Security Posture . . . 12
6.2 Before Deploying to a Public Portfolio . . . 12

7 Interview Presentation Guide **13**
7.1 The 30-Second Pitch . . . 13
7.2 Likely Interview Questions & Strong Answers . . . 13
7.2.1 “Why did you build this?” . . . 13
7.2.2 “Walk me through the architecture.” . . . 13
7.2.3 “How did you handle rate limits?” . . . 13
7.2.4 “How do you aggregate scores from three different APIs?” . . . 13
7.2.5 “What would you do differently?” . . . 13
7.2.6 “How is this different from just using the APIs directly?” . . . 14
7.3 Technical Keywords to Use Naturally . . . 14

8 Next Iterations **15**
8.1 Priority 1 – Production Readiness . . . 15
8.2 Priority 2 – Feature Expansion . . . 15
8.3 Priority 3 – Engineering Quality . . . 15
8.4 Priority 4 – Intelligence Layer . . . 16

9 Recommended Deployment Architecture **17**
9.1 Minimal Portfolio Deployment 17

10 Project Summary Card **18**

Executive Summary

ARES CTI Dashboard is a full-stack web application that aggregates **Cyber Threat Intelligence (CTI)** from three industry-standard APIs: **AbuseIPDB**, **AlienVault OTX**, and **VirusTotal**. It gives security analysts a single pane of glass to:

- Look up any **Indicator of Compromise (IOC)** — IP address, domain, or file hash — and receive an aggregated risk score (0–100) from multiple sources.
- Browse a live **threat intelligence feed** from AlienVault OTX subscribed pulses.
- Monitor the **health** of all integrated data sources in real time.
- Review raw API responses per source for deeper analyst investigation.

> **Project motivation:** Most CTI tools require switching between multiple browser tabs and platforms. ARES collapses that into one dashboard, normalises risk scores to a common 0–100 scale, and caches results so analysts are not burning through API rate limits on repeated lookups.

Key Numbers at a Glance

Dimension	Value	Notes
External APIs integrated	3	AbuseIPDB, AlienVault OTX, VirusTotal
IOC types supported	3	IP, Domain, File Hash
Bulk lookup limit	50 IOCs	Parallelised via ThreadPoolExecutor
Cache TTL (indicators)	60 min	Prevents redundant upstream calls
Cache TTL (feed)	5 min	Near-real-time pulse freshness
API endpoints	8	Health, indicators, bulk, feed, stats
Frontend pages	3	Dashboard, Lookup, Feed

What the App Does

Core Features

IOC Lookup

An analyst types an IP address (`1.2.3.4`), domain (`evil.com`), or file hash (`d41d8cd9...`) into the search bar. The app auto-detects the IOC type using regex, then fans out **parallel requests** to all relevant threat intelligence sources. The analyst sees:

- A **circular risk gauge** coloured green / amber / red
- Aggregated metadata: country of origin, last-seen timestamp, contributing sources
- Deduplicated **threat tags** across all sources
- A **per-source breakdown** panel
- Toggle to expose the **raw JSON** from each source

Threat Feed

The Feed page pulls from AlienVault OTX subscribed pulses – community-curated threat reports. The analyst sees a table of pulses with name, author, tags, indicator count, and creation date. The feed auto-refreshes every 5 minutes.

Dashboard Overview

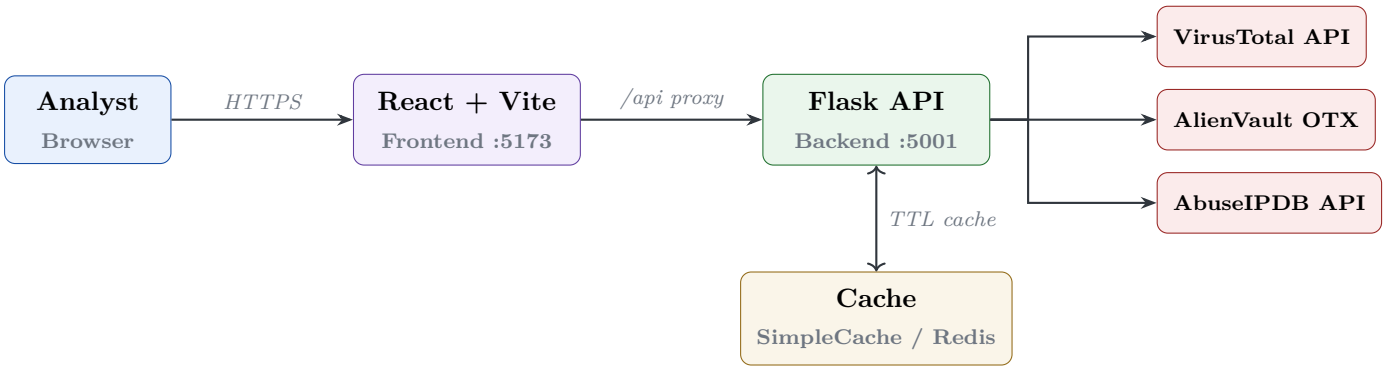
The landing page surfaces aggregated stats, a horizontal bar chart of the top 10 threat tags (Recharts), source health status, and a recent pulses summary table.

Bulk Lookup (API-first)

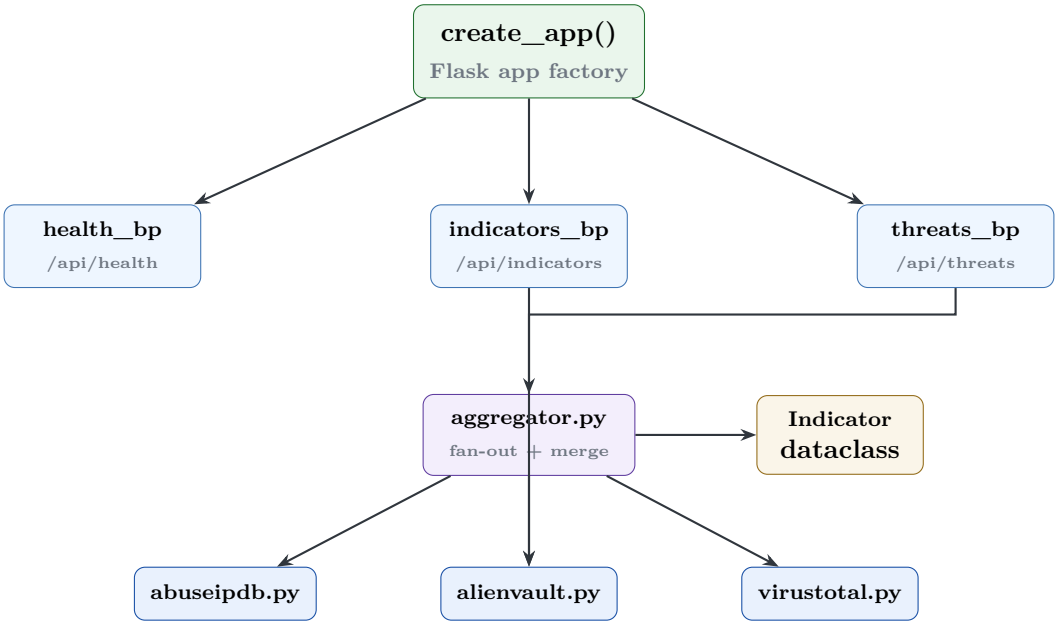
A POST endpoint (`/api/indicators/bulk`) accepts up to 50 IOCs and processes them in parallel threads, returning a map of IOC to result. No UI yet – ready for scripting and automation workflows.

System Architecture

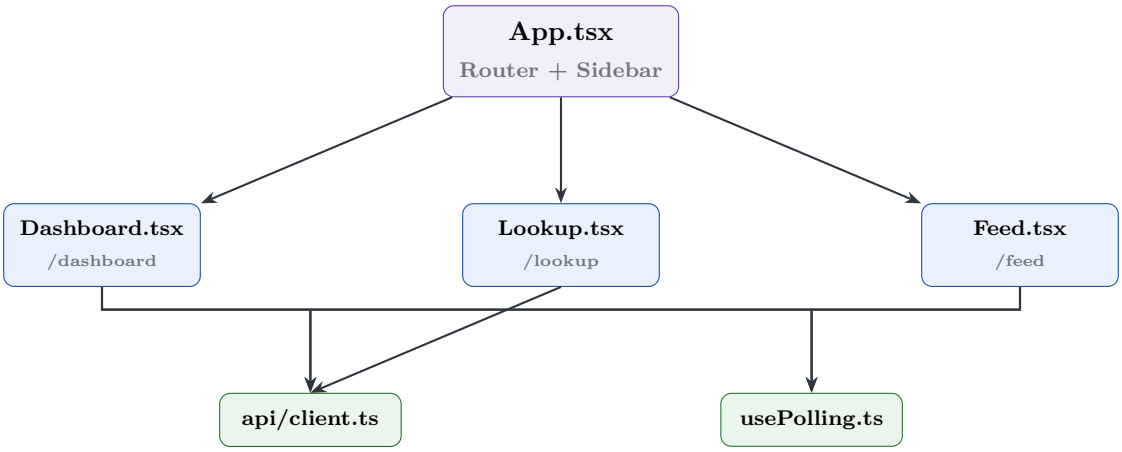
High-Level Architecture



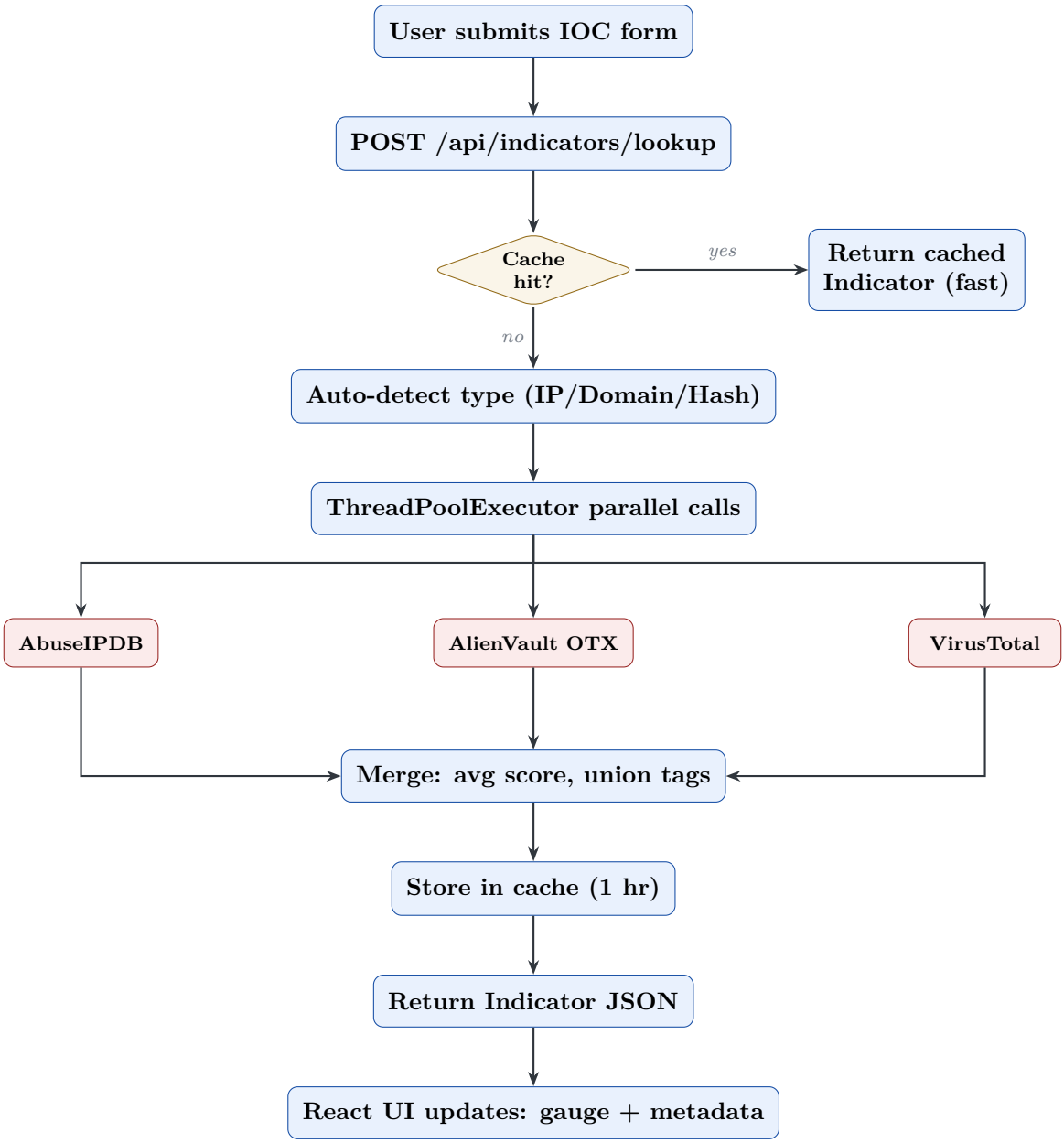
Backend Architecture – Flask Blueprints



Frontend Architecture – React Component Tree



IOC Lookup Data Flow



Caching Strategy

Cache Key Pattern	TTL	Backend (dev)	Backend (prod)
`indicator:{type}:{ioc}`	3600 s (1 hr)	SimpleCache	RedisCache
`/threats/feed?limit=N`	300 s (5 min)	SimpleCache	RedisCache
`/threats/stats`	600 s (10 min)	SimpleCache	RedisCache

Why these TTLs?

- **1 hour for indicators:** Threat intel for a given IOC rarely changes minute-to-minute. One hour reduces upstream API calls by ∼95% for repeated lookups.
- **5 minutes for feed:** Threat pulses are published continuously; a short TTL keeps the feed near-real-time.
- **10 minutes for stats:** Aggregated tag statistics only shift meaningfully when the feed itself changes.

Design Choices & Rationale

Backend: Why Flask?

> Flask was chosen for its **minimal footprint and explicit structure**. A CTI aggregator is fundamentally a thin proxy layer: call external APIs, merge results, cache. Flask's blueprint system maps cleanly onto this domain (one blueprint per resource group), and flask-caching plugs in without boilerplate.

Alternative considered: FastAPI would give async-native support and automatic OpenAPI docs. For I/O-bound fan-outs, async is natural – however, `ThreadPoolExecutor` achieves the same parallelism without requiring an async mental model throughout the codebase.

Parallelisation: ThreadPoolExecutor

The aggregator fires requests to all three sources simultaneously. Total latency ≈ max(slowest API) rather than their sum.

Approach	Latency (example)	Notes
Sequential (3 calls)	∼1.5 s	AbuseIPDB 0.4s + OTX 0.6s + VT 0.5s
Parallel (ThreadPoolExecutor)	∼0.6 s	Bounded by slowest source only

Score Normalisation

Each source expresses risk in a different format. The aggregator normalises to 0–100:

Source	Raw Signal	Normalisation
AbuseIPDB	Confidence score 0–100	Direct pass-through
VirusTotal	malicious / total engines	Multiply by 100
AlienVault OTX	Pulse count (threat reports)	`min(count * 10, 100)`
Aggregated	—	**Simple average of all source scores**

Frontend: Why Vite + React?

- **Vite**: Near-instant HMR; ES module-native bundler with zero config overhead.
- **React**: Component model maps to the dashboard paradigm – each data card is isolated.
- **TypeScript**: The API contract is enforced via interfaces mirroring the Flask `Indicator` dataclass. Type errors surface immediately.
- **Recharts**: Declarative chart API that composes with React's rendering model; responsive containers eliminate media-query boilerplate.

The Proxy Pattern

The Vite dev server proxies `/api/*` to `http://localhost:5001`. Benefits:

1. No CORS configuration needed in development.
2. Frontend code has no hardcoded backend URL – always references `/api`.
3. In production, Nginx sitting in front of both services proxies the same path.

App Factory Pattern (Flask)

The `create_app()` factory rather than a module-level app instance enables:

- **Multiple configurations**: dev/prod switch based on `FLASK_ENV`.
- **Testability**: tests can instantiate the app with a test config in isolation.
- **Clean blueprint registration**: each resource group registers itself explicitly.

API Reference

Endpoint Summary

Method	Path	Purpose	Cache TTL
GET	`/api/health`	Service health for all 3 sources	None
POST	`/api/indicators/lookup`	IOC lookup (auto-detect type)	3600 s
GET	`/api/indicators/ip/<ip>`	Shorthand IP lookup	3600 s
GET	`/api/indicators/domain/<d>`	Shorthand domain lookup	3600 s
GET	`/api/indicators/hash/<h>`	Shorthand hash lookup	3600 s
POST	`/api/indicators/bulk`	Bulk lookup (max 50 IOCs)	per-IOC
GET	`/api/threats/feed?limit=N`	AlienVault OTX pulse feed	300 s
GET	`/api/threats/stats`	Aggregated tag statistics	600 s

Indicator Response Schema

Listing 1: Indicator JSON returned by all lookup endpoints

```
{
  "ioc":        "8.8.8.8",
  "type":       "ip",
  "score":      15,
  "sources":    ["abuseipdb", "alienvault", "virustotal"],
  "tags":       ["DNS", "public_resolver"],
  "country":    "US",
  "last_seen": "2025-01-15T10:30:00",
  "raw": {
    "abuseipdb":  { ... },
    "alienvault": { ... },
    "virustotal": { ... }
  }
}
```

Security Review

Current Security Posture

Control	Status	Risk	Notes
API keys in `.env`	**OK**	Low	Not committed to git
`.env` in `.gitignore`	**OK**	Low	Excluded at root level
Request timeouts	**OK**	Low	10 s per upstream call
Bulk limit (50 IOCs)	**OK**	Low	Prevents obvious DoS
API authentication	**None**	Medium	Endpoints are open
CORS configuration	**None**	Medium	Any origin accepted
Input validation	**Minimal**	Low–Med	Regex type-detect only
Rate limiting	**None**	Medium	No per-IP throttling
HTTPS enforcement	**None**	Medium	HTTP only in dev
Debug / Werkzeug	**Dev only**	Medium	Must be off in production

> **Portfolio / demo use:** The current posture is acceptable for a localhost demo or a private-network deployment behind a firewall. It should **not** be exposed to the public internet without adding authentication and HTTPS.

Before Deploying to a Public Portfolio

1. **Add authentication** – Even a simple API token header check (`X-API-Key`) or HTTP Basic Auth via Flask-HTTPAuth prevents random internet traffic from burning your rate-limited free-tier API keys.
2. **Add CORS control** – Install `flask-cors` and whitelist only your portfolio's origin.
3. **Use HTTPS** – Terminate TLS at Nginx/Caddy in front of Flask. Never run Flask with `debug=True` on a public host.
4. **Add rate limiting** – `Flask-Limiter` with a Redis backend: e.g. 60 lookups/hour per IP.
5. **Validate input lengths** – Limit IOC string length (e.g. max 255 chars) before passing to services.
6. **Sanitise error messages** – Do not leak internal stack traces to callers; log server-side only.
7. **Use production WSGI server** – Replace `flask run` with `gunicorn`. Never use Werkzeug in production.
8. **Rotate API keys** – If you ever accidentally commit your `.env`, rotate all three keys immediately and audit git history: `git log -p | grep KEY`.

Interview Presentation Guide

The 30-Second Pitch

> "ARES CTI Dashboard is a full-stack threat intelligence aggregator I built from scratch. It combines data from three industry-standard APIs – AbuseIPDB, AlienVault OTX, and VirusTotal – into a single analyst-facing dashboard. You search for any IP, domain, or file hash; the app fans out parallel requests to all three sources, normalises the risk scores onto a common 0–100 scale, merges the metadata, and caches the result so you are not burning through API rate limits. The backend is Flask with a blueprint-per-resource architecture, and the frontend is React with TypeScript."

Likely Interview Questions & Strong Answers

"Why did you build this?"

"I wanted to understand how CTI tools work under the hood. Most analysts pivot between three or four browser tabs to cross-reference a suspicious IP. I wanted to collapse that into one interface and own the aggregation logic – normalising disparate score formats into a common number is a non-trivial problem."

"Walk me through the architecture."

Key talking points (refer to the diagrams in Section 3):

- Blueprints separate routing from business logic; services isolate each external API call.
- ThreadPoolExecutor gives I/O parallelism – latency is bounded by the slowest source, not their sum.
- Layered caching: 1 hr for indicators, 5 min for feed.
- Dev/prod config split via Flask app factory; SimpleCache in dev, Redis in prod.

"How did you handle rate limits?"

"Two layers: (1) caching means repeated lookups of the same IOC never hit external APIs – results are served from cache for up to an hour. (2) Bulk endpoint caps at 50 IOCs per request. For production I would add Flask-Limiter for per-IP throttling."

"How do you aggregate scores from three different APIs?"

"Each API expresses risk differently. AbuseIPDB gives a direct 0–100 confidence score. VirusTotal gives malicious-engine-count divided by total-engine-count – I multiply by 100. AlienVault gives a pulse count – I scale with `min(count * 10, 100)`. Then I average the three. It is a heuristic; a production system would use weighted averages based on source reliability, or a trained classifier."

"What would you do differently?"

- **No auth**: for a public deployment, I would add a token check and rate limiting.
- **Score model is naive**: simple average across sources with different base rates. I would research Bayesian score fusion.

- **No automated tests**: I have a `tests/` directory set up but only one health check test. I would add aggregator unit tests and API integration tests.
- **Components/ is empty**: shared card and table UI are not yet extracted into reusable components.

"How is this different from just using the APIs directly?"

- One request instead of three – fan-out handled server-side.
- Normalised, comparable score instead of three incompatible formats.
- Caching reduces API consumption significantly.
- Single UI for analysts who cannot curl APIs.
- Extensible: adding a 4th source means one new service file.

Technical Keywords to Use Naturally

Backend	Frontend	Domain
Flask app factory	React Router	IOC / Indicator of Compromise
Blueprint pattern	Custom hooks	Threat Intelligence (CTI)
ThreadPoolExecutor	Recharts	Score normalisation
Flask-Caching	TypeScript interfaces	API fan-out pattern
SimpleCache / Redis	Vite dev proxy	Cache TTL strategy
python-dotenv	StrictMode	Parallel I/O

Next Iterations

Priority 1 – Production Readiness

1. **Authentication**: `Flask-HTTPAuth` with a token header. Prevents burning free-tier API quota.
2. **HTTPS + reverse proxy**: Nginx or Caddy in front of Flask + Vite build. Free TLS via Let's Encrypt.
3. **Rate limiting**: `Flask-Limiter` + Redis – e.g. 120 lookups/hr per IP, 20 bulk/hr.
4. **WSGI server**: `gunicorn -w 4` with a `Procfile` or systemd unit.
5. **Input validation**: `pydantic` or `marshmallow` for request body schema validation.
6. **CORS**: `flask-cors` configured to allow only your portfolio domain.

Priority 2 – Feature Expansion

1. **Lookup history**: Store past lookups in SQLite/PostgreSQL via Flask-SQLAlchemy. Show recent IOCs with timestamps.
2. **Bulk lookup UI**: Frontend page for the existing `/api/indicators/bulk` endpoint – progress bar, results table, CSV export.
3. **IOC list upload**: Paste or upload a newline-separated `.txt` / `.csv` for bulk scan.
4. **Watchlist / alerting**: Background Celery job re-checks IOCs periodically; webhooks or email on score change.
5. **Domain WHOIS**: Add registration date, registrar, and nameservers via `python-whois`.
6. **MISP integration**: Pull indicators from a local MISP instance alongside the existing three sources.

Priority 3 – Engineering Quality

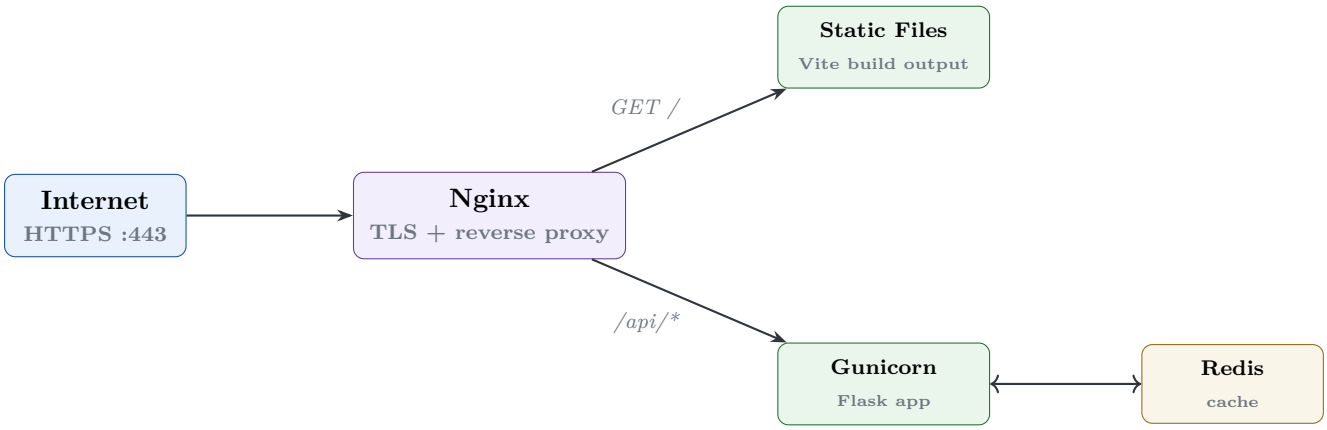
1. **Test suite**: Unit tests for aggregator merge logic; Flask test client integration tests per blueprint.
2. **Typed service layer**: Replace raw `dict` returns with `TypedDict` or Pydantic models.
3. **OpenAPI / Swagger**: Auto-generate with `flask-smorest` or `apiflask`.
4. **Docker Compose**: One `docker-compose up` spins backend, Nginx (serving Vite build), and Redis.
5. **Component library**: Extract card, table, and gauge into `src/components/` for reuse and Vitest coverage.
6. **CI/CD**: GitHub Actions – lint (Ruff / ESLint), type-check (mypy / tsc), tests, build on every push.

Priority 4 – Intelligence Layer

1. **Weighted score fusion**: Configurable source weight vector; research how commercial platforms (Recorded Future, ThreatConnect) weight source reliability.
2. **Graph visualisation**: `react-force-graph` to show relationships between IOCs – same AS, related domains, shared infrastructure.
3. **Threat actor attribution**: Cross-reference IOC tags with MITRE ATT&CK STIX dataset.
4. **STIX/TAXII export**: Allow export of found IOCs in STIX 2.1 format for SIEM/SOAR tools.
5. **ML scoring**: Train a simple classifier on labelled IOCs for a more calibrated risk score than the heuristic average.

Recommended Deployment Architecture

Minimal Portfolio Deployment



Recommended free-tier cloud options:

- **Railway.app** – Backend (Gunicorn) + Redis as separate services; Vite build on Vercel.
- **Render.com** – Web Service (Gunicorn) + Redis add-on. Spins down after 15 min of inactivity on free tier.
- **Fly.io** – Docker-based deployment; generous free tier for always-on demos.
- **VPS (DigitalOcean / Hetzner)** – $4–6/month; full control with Docker Compose + Nginx + Certbot.

Project Summary Card

ARES CTI Dashboard – Quick Reference

Type	Full-Stack Web Application
Backend	Python 3.12 / Flask 3.0+ / flask-caching / requests
Frontend	React 18 / TypeScript / Vite / React Router / Recharts
Ext. APIs	AbuseIPDB, AlienVault OTX, VirusTotal
Cache	SimpleCache (dev) / Redis (prod)
Pattern	App factory, Blueprint per resource, Service layer
Concurrency	ThreadPoolExecutor – parallel API fan-out
IOC types	IP address, Domain, File hash (MD5/SHA256)
Endpoints	8 REST endpoints
Pages	Dashboard, IOC Lookup, Threat Feed
Dev ports	Backend :5001 / Frontend :5173

Built by Rafael Rodriguez • April 2026
github.com/rodriguezgonzalez